

第5章 函数设计与使用

- 第一章已学习内容review
- 特别补充说明：return语句，形参与实参，默认值参数(函数名.__defaults__)
- 可变长度参数、仅允许关键字传递的形参（不作要求），参数传递的序列解包
- 变量作用域
- 案例精选

Review篇

以下Page3-Page13在第一章已经讲述

Review:函数

- 在实际开发中，有许多操作时完全相同或非常相似的，仅仅是要处理的数据不同，为此需要实现代码复用。
- 将可能需要反复执行的代码封装为函数，并在需要该功能的地方进行调用，不仅可以实现代码复用，更重要的是可以保证代码的一致性，只需要修改该函数代码则所有调用均受到影响。
- Python提供了许多常用的内置函数我们已学习print()、input()等
- 如何定义自己的函数呢？

函数定义

def 函数名([形参列表]):

'''注释'''

函数体

- ✓ 函数体的开始以**def**开始，然后是一个空格和函数的名称，接下来是一对圆括号
- ✓ 圆括号内是形式参数列表，称为**形参**，**形参**如果有多个以逗号(,)隔开，参数可以没有，也**必须**保留一对空的圆括号
- ✓ 括号后面的**冒号**必不可少
- ✓ 与其它语言不同，函数形参**不需要**声明类型，也**不需要**指定函数返回值类型
- ✓ Python**允许嵌套定义函数**

```
def fahrenheit_from_Celsius(Celsius):
```

```
    """摄氏度转换到华氏度"""
```

```
    Fahrenheit = (9 / 5) * Celsius + 32
```

```
    return Fahrenheit
```

```
temperature_cel = 35
```

```
temperature_fah = fahrenheit_from_Celsius(temperature_cel)
```

```
print(temperature_fah)
```

```
def my_sum(start, stop, step):
```

```
    """ 求和, 从start到stop (不包括) , 步长为step """
```

```
    i = start
```

```
    sum = 0
```

```
    while i < stop:
```

```
        sum += i
```

```
        i += step
```

```
    return sum
```

```
sumValue = my_sum(1, 10, 3)
```

```
print(sumValue)
```

代码在备注中

函数定义：文档字符串

def 函数名([形参列表]):

'''注释'''

函数体

- 定义函数（及模块）时，开头部分的**注释**并不是必需的，但是如果为函数的定义加上这段注释的话，可以为用户提供友好的提示和使用帮助。
- 该字符串保存在函数（及模块）对象的__doc__属性中，称为docstring
- help(func)可以查看相应的__doc__属性
- IDLE界面会在相应的calltip给出其注释

```
def fahrenheit_from_celsius(celsius):  
    """ 摄氏度转换到华氏度 """  
    fahrenheit = (9 / 5) * celsius + 32  
    return fahrenheit
```

```
>>> help(fahrenheit_from_celsius)  
Help on function fahrenheit_from_celsius in module __main__:  
  
fahrenheit_from_celsius(celsius)  
    摄氏度转换到华氏度  
  
>>> print(fahrenheit_from_celsius(  
    (celsius)  
    摄氏度转换到华氏度
```

函数定义：函数体

def 函数名([形参列表]):

'''注释'''

函数体

- 函数体一般包括多行，相对于def关键字必须保持一定的空格缩进，是函数执行的代码块
- 如果函数体比较简单时，也可以与def在同一行
- 函数体可以只写一句pass语句为空语句，经常用在定义一个空函数，以后再扩充
- 函数可以返回值，也可以不返回。
 - 通过return语句返回对象
 - 如果函数体中包含return语句，不论return语句出现在函数的什么位置，一旦得到执行将直接结束函数的执行（即后面的代码不再执行），并同时返回return语句中相应表达式所创建的对象；如果没有return，返回的值为None，其类型为NoneType（只有一个值即None）
 - 如果执行到函数结束也无return语句，则等价于 return None

```
def fahrenheit_from_celsius(celsius):  
    """ 摄氏度转换到华氏度 """  
    fahrenheit = (9 / 5) * celsius + 32  
    return fahrenheit  
  
def cubic(n): return n**3  
  
def printName(name):  
    """ 将姓名打印出来 """  
    print("我的名字是:", name)
```

函数：函数调用

- 函数调用：

函数名([实参列表])

fahrenheit_from_Celsius(temperature_cel)

```
def fahrenheit_from_Celsius(Celsius):  
    """摄氏度转换到华氏度"""  
    Fahrenheit = (9 / 5) * Celsius + 32  
    return Fahrenheit
```

```
temperature_cel = 35  
temperature_fah = fahrenheit_from_Celsius(temperature_cel)  
print(temperature_fah)
```

- 实参(argument)：应该与形参（函数定义时的参数）对应，如有多个参数之间以逗号分隔，表示传递实参给出的对象引用到相应的形参，即函数调用时向其传递实参，python采用赋值传递(pass by assignment)的策略

形参变量=实参变量，即形参变量与实参变量指向同一个对象

- 绝大多数情况下，在函数内部直接修改形参的值不会影响实参
 - 对于不可变对象，显然如此
 - 对于可变对象，需要小心处理
- 为了避免side effect，函数中的形参（变量）的使用范围为函数体(作用域部分会详细介绍)

位置参数

- 位置参数（positional arguments）是比较常用的形式，调用函数时实参和形参的顺序必须严格一致，并且实参和形参的数量必须相同。

```
def my_sum(start, stop, step):  
    """ 求和, 从start到stop (不包括), 步长为step """  
  
    i = start  
    sum = 0  
    while i < stop:  
        sum += i  
        i += step  
    return sum  
  
sumValue = my_sum(1, 10, 3)  
print(sumValue)
```

```
>>> def demo(a, b, c):  
    print(a, b, c)  
>>> demo(3, 4, 5)                #按位置传递参数  
3 4 5  
>>> demo(3, 5, 4)  
3 5 4  
>>> demo(1, 2, 3, 4)            #实参与形参数量必须相同  
TypeError: demo() takes 3 positional arguments but 4  
were given
```

默认值参数

- 仅仅包含形参名的参数称为**位置 (positional) 参数**, 根据其出现的位置顺序来逐个匹配相应的实参
- Python在定义函数时, 支持**默认值参数**, 即可为形参设置默认值(缺省值)。

```
def 函数名(……, 形参名=默认值):  
    函数体
```

- 调用带有**默认值参数**的函数时, 可以不对默认值参数进行赋值, 也可以赋值, 具有较大的灵活性。
- 使用 “**函数名.__defaults__**” 以元组的形式查看函数所有默认值参数的当前值。

```
>>> def say( message, times =1 ):  
        print (message * times)
```

```
>>> say('hello')#不为默认值参数传值  
hello
```

```
>>> say('hello', 3)#使用调用者显示传递的值  
hellohellohello
```

```
>>> say('hi', 7)#使用调用者显示传递的值  
hihihihihihihihihihi
```

```
>>> say.__defaults__  
(1,)
```

默认值参数(defaults parameter)

- 函数定义时最前面是位置参数，然后是默认值参数
- 即默认值参数右边不能再有位置参数。
- 为什么这样设计？调用时: 有默认值的参数可以省略; 这样才好匹配：首先普通的位置参数必须的进行匹配，剩下的如果实参有，则采用实参的值; 如果没有,则采取默认值

```
>>> def f(a=3,b,c=5):  
    print a,b,c
```

```
SyntaxError: non-default argument follows default argument
```

```
>>> def f(a=3,b):  
    print a,b
```

```
SyntaxError: non-default argument follows default argument
```

```
>>> def f(a,b,c=5):  
    print a,b,c
```

```
>>>
```

默认值参数(defaults parameter):示例

```
def my_sum(start, stop, step):  
    """ 求和, 从start到stop (不包括), 步长为step """
```

```
def my_sum_default(start, stop, step=1, init=0):  
    """ 求和, 从start到stop (不包括), 步长为step, 缺省为1, 和最后再加上init, 缺省为0 """
```

```
    i = start  
    sum = init  
    while i < stop:  
        sum += i  
        i += step  
    return sum
```

```
print(my_sum_default(0, 101, 1, 0))  
print(my_sum_default(0, 101, 1))  
print(my_sum_default(0, 101))  
print(my_sum_default(0, 101, 1, 1000))  
print(my_sum_default(0, 101, 2))
```

如果step取默认值, 而想改变init默认值, 怎么办?

关键字参数(keyword argument)

- 关键字参数主要指**实参**，即**调用函数时**的参数传递方式。
- 通过**关键字参数可以按参数名字**传递值，实参顺序可以和形参顺序不一致，但不影响传递结果，避免了用户需要牢记位置参数顺序的麻烦。
- 注意**关键字参数**与函数定义时的**缺省值参数**的区别，虽然两者形式一致，但两者之间**并不等同**：**一个是在函数调用时，一个是在函数定义的时候**
- 调用时可以使用位置参数和关键字来传递参数，但是与函数定义时类似，**关键字参数右边不能有位置参数**

```
>>> def demo(a, b, c=5):  
        print (a, b, c)  
>>> demo(3, 7)  
3 7 5  
>>> demo(a=7, b=3, c=6)  
7 3 6  
>>> demo(c=8, a=9, b=0)  
9 0 8
```

关键字参数(keyword argument)

- 在函数定义中有多个缺省值参数时

通过关键字参数来明确传递哪些参数，哪些采用默认值

```
def func(a, b, c=5, d=6): print(a, b, c, d)
func(1, 2, 7)
func(1, 2, 5, 8)
func(b=1, a=2, c=7)
func(1, 2, d=8)
func(1, b=2, d=8)
```

```
def my_sum_default(start, stop, step=1, init=0):
    """ 求和, 从start到stop (不包括), 步长为step,
        缺省为1, 和最后再加上init, 缺省为0 """
    i = start
    sum = init
    while i < stop:
        sum += i
        i += step
    return sum

print(my_sum_default(0, 10, init=100))
```

特别补充说明：return语句

- 在调用内置数据类型的方法时，一定要注意该方法有没有返回值。
- `print([1,2,3,4,9,5,7].append(8)) print([1,2,3,4,9,5,7].sort())`可能不是想要的结果：
 - `sort`、`append`方法为原地排序，返回`None`

```
>>> def maximum( x, y ):
        if x>y:
            return x
        else:
            return y
```

```
>>> theMax = maximum(3, 4)
>>> print(theMax)
4
```

(1)具有返回值

```
>>> def maximum( x, y ):
        if x>y:
            print(x)
        else:
            print(y)
```

```
>>> theMax = maximum(3, 4)
4
>>> print(theMax)
None
```

(2) 没有返回值。

特别补充说明：形参与实参

- 可变对象的原地操作会影响函数外部实参的值，许多时候会要利用该特性

```
>>> def modify(v):  
#修改列表元素值  
    v[0] = v[0]+1
```

```
>>> a = [2]  
>>> modify(a)  
>>> a  
[3]
```

```
>>> def modify(v, item):  
#为列表增加元素  
    v.append(item)
```

```
>>> a = [2]  
>>> modify(a, 3)  
>>> a  
[2, 3]
```

```
>>> def modify(d):  
#修改字典元素值或为字典增加元素  
    d['age'] = 38  
>>> a = {'name': 'Dong',  
         'age': 37, 'sex': 'Male'}  
>>> a  
{'sex': 'Male', 'age': 37,  
 'name': 'Dong'}  
>>> modify(a)  
>>> a  
{'sex': 'Male', 'age': 38,  
 'name': 'Dong'}
```

注意：如果传递给函数的是Python可变序列，并且在函数内部使用下标或其他方式为可变序列增加、删除元素或修改元素值时，可以使得实参得到相应的修改。

特别补充说明：默认值参数

- 在函数定义时，计算默认值得到一个tuple，然后保存在 `func.__defaults__`
- 函数调用时如何确定默认值参数的值？
 - 如果实参中给出了相应的值，则该形参指向实参所引用的对象
 - 如果实参没有传递值，则该形参变量赋予默认值，而默认值保存在 `func.__defaults__`
 - 如果默认值参数的默认值为 **可变对象**，且前面调用该函数时原地修改了该对象，保存在 `__defaults__` 元组中的 **默认值也会被修改**
 - 这种特性可能导致很难发现的逻辑错误。
 - 也可能是故意这样来设计在函数的多次调用间 **保存状态信息**

5.3.1 默认值参数

```
def default_mutable(newitem, old_list=[]):  
    old_list.append(newitem)  
    return old_list
```

```
def test_default_mutable():  
    print (default_mutable.__defaults__)  
    print (default_mutable('5', [1, 2, 3, 4]))  
    print (default_mutable('aaa', ['a', 'b']))  
    print (default_mutable('a'))  
    print (default_mutable.__defaults__)  
    print (default_mutable('b'))  
    print (default_mutable.__defaults__)
```

```
def default_mutable2(newitem, old_list=None):  
    if old_list is None:  
        old_list=[]  
    old_list.append(newitem)  
    return old_list
```

```
([],)  
[1, 2, 3, 4, '5']  
['a', 'b', 'aaa']  
['a']  
(['a'],)  
['a', 'b']  
(['a', 'b'],)
```

注意：多次调用函数并且不为默认值参数传递值时，默认值参数值在第一次调用时被更改，可以使用函数名.__defaults__查看默认参数的**当前值**

缺省值参数的缺省值避免采用可变对象，或者采用可变对象但是函数体不要进行原地操作。

除非故意设计以利用可变对象的这一特征。在函数的多次调用间
保存状态信息

5.3 参数类型

- Python函数的定义非常灵活，在定义函数时不需要指定参数的类型，形参的类型完全由调用者传递的实参类型以及Python解释器的理解和推断来决定，类似于重载和泛型(polymorphism)；
- 函数编写如果有问题，只有在调用时才能被发现，传递某些参数时执行正确，而传递另一些类型的参数时则出现错误。
- 在Python中，函数参数有很多种：
 - 普通（位置）参数✓
 - 默认值参数（定义时）✓
 - 关键参数（调用时）✓
 - 可变长度参数等等。

5.3.3 可变长度参数 (variable-length parameters)

- **可变长度参数**在定义函数时主要有两种形式：

- `*parameter`：将那些**尚没有匹配形参的位置实参**组合成一个**元组**赋值给可变长度位置参数。
- `**parameter`：将那些**尚没有匹配形参的关键字实参**组合成一个**dict**赋值给该参数

- 如果存在可变长度位置参数，`*name`形式的参数应该出现在默认值参数之后
- 如果存在可变长度关键字参数，`**name`形式的参数应该出现在最后
- 每种类型的可变长度参数只能出现一次

```
def variable_parameter(a,b=4,*x, **y):  
    return (a, b, x, y)  
  
def test_variable_parameter():  
    print(variable_parameter(1))  
    print(variable_parameter(1,2))  
    print(variable_parameter(1,2,3,4,5))  
    print(variable_parameter(1,u=11,v=12,w=13))  
    print(variable_parameter(1,2,3,4,5,u=11,v=12,w=13))
```

```
(1, 4, (), {})  
(1, 2, (), {})  
(1, 2, (3, 4, 5), {})  
(1, 4, (), {'u': 11, 'v': 12, 'w': 13})  
(1, 2, (3, 4, 5), {'u': 11, 'v': 12, 'w': 13})
```

仅允许关键字传递的形参 (keyword-only parameter) 补充, 不作要求

- Python3还引入了**keyword-only形参**, 即这些参数**只能通过关键字参数**的形式来传递

```
def konly(a, *b, c, d=5):  
    return (a,b,c,d)
```

- keyword-only参数出现在可变长度的位置参数之后

```
def konly2(a, b, *, c, d=5):  
    return (a,b,c,d)
```

- 如果没有可变长度位置参数, 则添加一个*

- keyword-only参数也可以指定默认值, 即支持name或者name=default方式

```
print(konly(1, 2, c=3))  
print(konly2(1, 2, c=3, d=4))
```

- 调用时必须**采取关键字参数传递**, 或者**不传递**表示采用默认值

5.3.3 可变长度参数（教材上例子）

- 几种不同类型的参数可以混合使用
- ~~不建议这样做,易导致代码混乱而严重降低可读性,可能导致查错非常困难~~

```
>>> def func_4(a, b, c=4, *aa, **bb):  
    print (a, b, c)  
    print (aa)  
    print (bb)  
  
>>> func_4(1, 2, 3, 4, 5, 6, 7, 8, 9, xx=' 1' , yy=' 2' , zz=3)  
1 2 3  
(4, 5, 6, 7, 8, 9)  
{ 'zz' : 3, 'xx' : ' 1' , 'yy' : ' 2' }  
  
>>> func_4(1, 2, 3, 4, 5, 6, 7, xx=' 1' , yy=' 2' , zz=3)  
1 2 3  
(4, 5, 6, 7)  
{ 'zz' : 3, 'xx' : ' 1' , 'yy' : ' 2' }
```

形参与实参之间的匹配

- 函数定义中的形参顺序

位置参数 → 默认值参数 → 可变长度位置参数 → 仅允许关键字传递参数 → 可变长度字典参数

- 函数调用中的实参顺序: **位置参数 → 关键字参数**
- 函数调用时, 根据传递的实参按照以下规则匹配形参:
 - 首先按照**位置匹配**形参中的位置参数和默认值参数
 - 剩下的**位置参数组成tuple**赋值给**可变长度位置参数**
 - 接下来根据**关键字参数的名字匹配**形参中的各个参数
 - 剩下的**关键字参数组成dict**赋值给**可变长度字典参数**
 - 函数定义中**尚未匹配的参数**设置为**缺省值**
 - 如果仍然有尚未匹配的形参和实参则报错
 - 每个参数只能匹配一次

5.3.4 参数传递的序列解包

- 为含有多个变量的函数**传递参数**时，可以使用**Python列表、元组、集合、字典**以及其他可迭代对象作为实参，并在实参名称前加一个**星号***，Python解释器将自动进行解包。
 - 等价于传递多个**位置参数**，每个参数为迭代对象的每个元素（对于字典而言为每个key）
- ******表示对于**字典对象**进行序列解包
 - 要求字典对象的**key为字符串**
 - 等价于传递多个**关键字参数**，关键字的变量名为字典对象dictobj的key，而值为dictobj[key]
- 注意**序列解包**与函数定义时的**可变长参数**的区别。

```
>>> def demo(a, b, c):  
        print (a+b+c)  
  
>>> seq=[1, 2, 3]  
>>> demo(*seq)  
6  
  
>>> tup=(1, 2, 3)  
>>> demo(*tup)  
6  
  
>>> dic={1:'a', 2:'b', 3:'c'}  
>>> demo(*dic)  
6  
  
>>> set1={1, 2, 3}  
>>> demo(*set1)  
6  
  
>>> demo(*dic.values())  
abc  
  
>>> dic2 = {'a':1, 'b':2, 'c':3}  
>>> demo(**dic2)  
6
```

矩阵转置（第2章的内容）

- 使用列表推导式实现矩阵转置

```
>>>matrix = [ [1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]
```

```
>>> [ [ row[i] for row in matrix ] for i in range(4) ]
```

```
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

- 也可以使用内置函数来实现矩阵转置

```
>>>list(zip(*matrix))          #序列解包zip( [1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12])
```

```
[(1, 5, 9), (2, 6, 10), (3, 7, 11), (4, 8, 12)]
```

5.5 变量作用域

5.5 变量作用域(scope)

- 变量保存的是对象的引用:
 - **命名空间 (namespace)** 是名字 (变量) 和对象的映射, python采用dict来实现
 - 命名空间中的名字 (变量) 可以访问的范围称为 (命名空间的) **作用域**。
 - 对象没有所谓作用域的概念, 但是要引用对象, 必须要通过变量
- 一个变量在函数外部定义和在函数内部定义, 其作用域是不同的:
 - **局部变量**: 一般而言, 在函数内部定义 (包括参数) 的变量, 有效范围为函数体
 - **全局变量**: 在模块内定义且不在函数内定义的变量, 有效范围为定义的模块
 - 只有**定义 (赋值)**、**global**和**nonlocal**才会建立或者改变变量的作用域(namespace)
- 局部变量的引用比全局变量速度快。
- 全局变量增加了不同函数之间的隐式耦合度。

5.5.1 局部变量

- **局部变量**在函数内定义，只在该函数内起作用
- 函数的形参属于**局部变量**
- 函数结束时，无法再访问局部变量。

```
def local (a=0) :  
    x = 3  
    y = 4  
    print(a, x, y)
```

```
local (2)  
print(x)
```

```
2 3 4
```

```
Traceback (most recent call last):
```

```
print(x)
```

```
NameError: name 'x' is not defined
```

5.5.2 全局变量

- **全局变量**：其作用域为全局的，能够同时作用于函数内外。

```
def local(a=0):  
    x = 3  
    y = 4  
    print(a, x, y)
```

```
local(2)  
print(x) ✖
```

2 3 4

Traceback (most recent call last):
print(x)
NameError: name 'x' is not defined



```
def func1(a=0):  
    x = 3  
    y = 4  
    print(a, x, y)  
    print(X)
```

```
X = 88 # global  
print(X)  
func1(2)
```

88
2 3 4
88

自由变量：在函数体内未赋值但是使用的变量

函数定义时并不知道变量绑定的对象

函数调用时才确定变量绑定的对象

- **变量绑定**：决定变量所指向的对象
- **late binding**：函数中的自由变量在调用时才会绑定

变量的作用域搜索（只要求掌握LG规则）

- 在**访问**变量(注意不是**赋值，为自由变量**)时，怎么找到具体是哪个作用域内的变量？

- LEGB规则：**

- 按照顺序搜索相应的作用域，直到找到匹配为止
- Local → Enclosing → Global → Builtin

- global x**: 改变x的**作用域为全局变量**

- nonlocal x**: python3引入，表示x不是局部变量，而是**外层函数**中定义的变量，跳过local作用域从**里到外往外层函数（但不会到函数外）**搜索(不作要求)

Builtin(python)：在python环境的内置模块built-in中定义的变量

Global(模块)：在函数外部定义的变量或者在函数体内通过**global**声明过

Enclosing: 外层函数内部赋值的变量，从最里层往外层搜索

Local: 函数内部赋值的变量，且没有通过**global/nonlocal**声明过

赋值改变作用域

注意：

- global/nonlocal会跳过局部作用域，改变其作用域分别为全局范围和外层的某个函数范围。
- 赋值会改变变量的作用域，而赋值发生的地方决定了其所在的作用域

```
def func1():  
    print(X)
```

```
def func2():  
    print(X)  
    X = X + 1  
    print(X)
```

```
X = 88  
func1()  
print()  
func2()  
print()
```

88

Traceback (most recent call last):

print(X)

UnboundLocalError: local variable 'X' referenced before assignment

X在函数体内赋值，作用域为Local

赋值改变作用域

```
def func1():  
    print(X)  
  
def func2():  
    print(X)  
    X = X + 1  
    print(X)
```

给X赋值，X
为局部变量

88

UnboundLocalError: local
variable 'X'

```
def func2_():  
    print(X)  
    Y = X + 1  
    print(Y)
```

没有给X赋值，X是
自由变量，根据
LEGB规则，仍然为
全局变量

```
X = 88  
func2_()
```

88
89

```
def func3():  
    X = 1  
    print(X)  
  
X = 88  
func3()  
print(X)
```

给X赋值，X是局部
变量，内层的（局部）
变量会隐藏更外层同
名的（全局）变量

1
88

```
def func3_():  
    X = 1  
    print(X)  
    import sys  
    print(sys.modules[__name__].X)  
  
X = 88  
func3_()  
print(X)
```

1
88
88

可通过模块对象来访问全局变量

global改变作用域

- global x: 表示x为全局变量，当函数体要改变全局变量的值时使用

给X赋值，X为局部变量

```
def func2():  
    print(X)  
    X = X + 1  
    print(X)
```

```
X = 88  
func2()
```

UnboundLocalError: local
variable 'X'

没有给X赋值，X仍然为全局变量

```
def func2_():  
    print(X)  
    Y = X + 1  
    print(Y)
```

```
X = 88  
func2_()  
print(X)
```

88
89
88

X声明为全局变量

```
def func4():  
    global X  
    print(X) # global X  
    X = X + 1  
    print(X)
```

```
X = 88  
func4()  
print(X)
```

88
89
89

函数作用域(补充, 不作要求)

- 可以在任何地方 (选择、循环、函数内部) 定义函数, 只要该处支持语句
- 选择和循环结构不会改变变量的作用域: 在退出循环之后仍然有效
- 列表解析式中引入的变量是局部变量, 退出之后不再有效
- **late binding**: 函数中的全局变量在调用时才会绑定

函数定义时缺省值对象n被保存在函数对象的属性__defaults__中

定义3个函数, 将三个函数对象保存在列表中

```
functions = []
for n in [1, 2, 3]:
    def func(x):
        return n*x
    functions.append(func)
```

函数定义时n
并没有绑定

```
functions = []
for n in [1, 2, 3]:
    def func(x, n=n):
        return n*x
    functions.append(func)

print([function(2) for function in functions])
```

#期待输出 [2,4,6], 但是输出 [6,6,6]
`print([function(2) for function in functions])`

函数定义时n绑定为循环结束时的值3

5.7 案例精选

5.7 案例精选

- 例1：编写函数计算圆的面积。

内置函数: `isinstance(obj, type)`
判断obj的类型是否是type

```
from math import pi as PI
#import types

def CircleArea(r):
    if isinstance(r, int) or isinstance(r, float): # 确保接收的参数为数值
        return PI * r * r
    else:
        print('You must give me an integer or float as radius.')

print(CircleArea(3))
```

28.274333882308138

5.7 案例精选

- 例2：编写函数，接收任意多个实数，返回一个元组，其中第一个元素为所有参数的平均值，其他元素为所有参数中大于平均值的实数。

```
def demo2(*para):  
    avg = sum(para) / len(para)  
    # 注意Python 2.x与Python 3.x对除法运算符"/"的解释不同  
    g = [i for i in para if i > avg]  
    return (avg,) + tuple(g)  
  
print(demo2(1, 2, 3, 4))
```

(2.5, 3, 4)

元组的+运算

5.7 案例精选

- 例3：编写函数，接收字符串参数，返回一个元组，其中第一个元素为大写字母个数，第二个元素为小写字母个数。

```
def demo3(s):  
    result = [0, 0]  
    for ch in s:  
        if 'a' <= ch <= 'z':  
            result[1] += 1  
        elif 'A' <= ch <= 'Z':  
            result[0] += 1  
    return result
```

```
print(demo3('aaaabbbbC'))
```

ch.islower()
ch.isupper()

[1, 8]

5.7 案例精选

- 例4：编写函数，接收包含20个整数的列表lst和一个整数k作为参数，返回新列表。将lst中元素**循环左移k位**，移出的元素转移到lst后部。
- 处理规则为：将列表lst中下标k之前的元素逆序，下标k之后的元素逆序，然后将整个列表lst中的所有元素逆序。

```
def demo4(lst, k):
```

```
    x = lst[:k]
```

```
    x.reverse()
```

```
    y = lst[k:]
```

```
    y.reverse()
```

```
    r = x + y
```

```
    r.reverse()
```

```
    return r
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]  
[6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 1, 2, 3, 4, 5]
```

```
lst = list(range(1, 21))
```

```
print(lst)
```

```
print(demo4(lst, 5))
```

5.7 案例精选

- 例5：编写函数，接收整数参数t，返回斐波那契数列中大于t的第一个数。

1	1	2	3		prev	curr		
fib(1)	fib(2)	fib(3)	fib(4)	...	fib(n-2)	fib(n-1)	fib(n)	fib(n+1)
						fib(n-1)	fib(n)	
						prev	curr	

```
def demo5(t):  
    a, b = 1, 1  
    while b <= t:  
        a, b = b, a + b  
    else:  
        return b
```

```
print(demo5(50))
```

55

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

```
def fib_demo5(t):  
    """ 返回小于t的最大fibonacci数 """  
    prev, curr = 1, 1  
    while curr <= t:  
        prev, curr = curr, prev + curr  
    else:  
        return curr
```

5.7 案例精选

知识点：

min / enumerate/ operator + / random.randint / list comprehension

例6：编写函数，查找整数列表lst中的最小值及其下标(可能有多)。接收一个包含若干整数的列表参数lst，返回一个元组，其中第一个元素为列表lst中的最小值，其余元素为最小值在列表lst中的下标（最小值出现的位置）

```
import random
def demo6(lst):
    m = min(lst)
    result = (m,)
    for index, value in enumerate(lst):
        if value == m:
            result = result + (index,)
    return result
```

```
x = [random.randint(1, 20) for i in range(50)]
print(x)
print(demo6(x))
print(demo6_2(x))
```

```
[12, 6, 6, 13, 2, 5, 2, 11, 18, 12, 7, 14, 6, 2, 17, 5, 11, 19, 18, 5, 7, 19, 14,
, 10, 17, 2, 5, 13, 16, 1, 1, 2, 17, 17, 14, 16, 4, 10, 20, 15, 8, 4, 17, 13, 17
, 9, 10, 15, 10, 15]
(1, 29, 30)
```

```
def demo6_2(lst):
    min_value = min(lst)
    result = [min_value]
    for index, value in enumerate(lst):
        if value == min_value:
            result.append(index)
    return tuple(result)
```

建议使用list，而不是tuple来保存中间结果

注意：你的运行结果可能不一样！

5.7 案例精选

- 例7：编写函数，接收一个整数t为参数，打印杨辉三角形前t行。

```
def demo7(t):  
    print([1])      # 第一行  
    print([1, 1])   # 第二行  
    line = [1, 1]  # line纪录上一行的内容  
    for i in range(2, t): # 从第3行开始，每次输出一行  
        r = []      # r = [1]  
        # 根据上一行计算下一行中间部分的内容  
        for j in range(0, len(line) - 1):  
            r.append(line[j] + line[j + 1])  
        line = [1] + r + [1] # r.append(1); line = r  
    print(line)
```

demo7(10)

下一行相比上一行多了一个数，最前面和最后为1，中间的各项都是上一行中连续两项之和

```
[1]  
[1, 1]  
[1, 2, 1]  
[1, 3, 3, 1]  
[1, 4, 6, 4, 1]  
[1, 5, 10, 10, 5, 1]  
[1, 6, 15, 20, 15, 6, 1]  
[1, 7, 21, 35, 35, 21, 7, 1]  
[1, 8, 28, 56, 70, 56, 28, 8, 1]  
[1, 9, 36, 84, 126, 126, 84, 36, 9, 1]
```

line: **[1, 4, 6, 4, 1]**

→ **[1, 5, 10, 10, 5, 1]**

5.7 案例精选（自学，不讲）

例8：编写函数，验证Goldbach猜想: 大于2的偶数都可写成两个质数之和

接收一个正偶数为参数，输出两个素数，并且这两个素数之和等于原来的正偶数。如果存在多组符合条件的素数，则全部输出。

```
import math
def IsPrime(n):
    m = int(math.sqrt(n)) + 1
    for i in range(2, m):
        if n % i == 0:
            return False
    return True

def demo8(n):
    if isinstance(n, int) and n > 0 and n % 2 == 0:
        for i in range(3, int(n / 2) + 1):
            if IsPrime(i) and IsPrime(n - i):
                print(i, '+', n - i, '=', n)

demo8(60)
```

- 一个正整数 n 分解成两个正整数之和，其中一个数一定小于等于 $n/2$
- 一个正整数 n 分解成两个正整数之积，其中一个数一定小于等于 $\sqrt{n}$
- **素数或质数(prime number)**
 - 大于1的自然数中，除了1和它本身以外不再有其他因数
- **如何判断两个数互素或互质？**
 - 两个数没有公因子

```
7 + 53 = 60
13 + 47 = 60
17 + 43 = 60
19 + 41 = 60
23 + 37 = 60
29 + 31 = 60
```

5.7 案例精选（自学，不讲）

<https://baike.baidu.com/item/最大公约数>
<https://baike.baidu.com/item/最小公倍数>

例9：编写函数，求两个正整数的最大公约数和最小公倍数

接收两个正整数(比如24,60)作为参数，返回一个数组，其中第一个元素为最大公约数，第二个元素为最小公倍数。

```
def demo9(m, n):  
    if m > n:  
        m, n = n, m  
    p = m * n  
    while m != 0:  
        r = n % m  
        n = m  
        m = r  
    return (n, int(p / n))  
print(demo9(30, 20))
```

```
def demo9_(m, n):  
    if m > n:  
        m, n = n, m  
    p = m * n  
    while m != 0:  
        m, n = n % m, m  
    return n, p // n
```

- **最大公约数**(最大公因数或最大公因子) greatest division factor：两个或者多个自然数的共有的因数中最大的一个
- **最小公倍数**(Least Common Multiple)：两个或者多个自然数的共有的倍数中除0之外的最小的一个
- **最大公因数和最小公倍数之间的性质**：两个自然数的乘积等于这两个自然数的最大公约数和最小公倍数的乘积
- **质因数分解法**：把每个数分别分解质因数
 - 最大公约数：把各数中的全部公有质因数提取出来连乘所得的积
 - 最小公倍数：把各数中的全部公有的质因数和独有的质因数提取出来连乘所得的积
- **辗转相除法（欧几里德法）求最大公约数**
 $\text{gcd}(m, n) = \text{gcd}(n \% m, m)$
...
(m=0, gcd)
- 两个自然数分别除以它们的最大公约数，所得的商是互质数（即最大公因数为1）

5.7 案例精选（自学，不讲）

例10：编写函数，接收一个所有元素值互不相等的整数数列x,和一个整数n，要求将值n的元素作为支点，将列表中所有值小于n的元素全部放到n前面，所有值大于n的元素放到n后面。

```
import random
def demo10(x, n):
    if n not in x:
        print(n, 'is not an element of', x)
        return

    i = x.index(n)  # 获取n在列表x的索引
    x[0], x[i] = x[i], x[0]  # n与第0个元素互换
    key = x[0]

    i = 0
    j = len(x) - 1
    while i < j:
        while i < j and x[j] >= key:  # 从后往前寻找第一个比指定元素小的元素
            j -= 1
        x[i] = x[j]

        while i < j and x[i] <= key:  # 从前往后寻找第一个比指定元素大的元素
            i += 1
        x[j] = x[i]
    x[i] = key
```

```
x = list(range(1, 10))
random.shuffle(x)
print(x)
demo10(x, 4)
print(x)
```

```
[2, 8, 1, 9, 5, 7, 4, 3, 6]
[3, 2, 1, 4, 5, 7, 9, 8, 6]
```

思考题：你能理解该程序实现的算法步骤吗？重点在i和j的具体含义是什么

可以采用IDLE调试器单步跟踪